State Effects

Design 1: Abstract CreatureState class that the specific states inherit from, implementing onEnter(), onExit(), checkTransitions() and getAction() methods

Pros	Cons
Follows SRP as each state class is responsible for its own logic, transitions and entry/exit effects	There are more classes to navigate
Follows OCP as adding a new state does not require any changes to existing states or any actor classes	

Design 2: Have all of the logic occur as conditionals inside playTurn() logic for each creature

Pros	Cons
The decision logic is visible in one place	Hard to read and maintain as there is an increase in the number of states
	Modifying and adding states requires directly editing the actor class

Status Effects

Design 1: Each effect implements the Status interface

Pros	Cons
Follows SRP as each class is responsible for its own logic	
Actor does not need to manage any behaviour of the effect	
Reusable as the statuses can be applied by any state to any actor without having	

to copy and paste logic	

Design 2: Have the effects occur directly on the actor and handle them inside playTurn()

Pros	Cons
No need to create new status classes for simple implementations	Actor has the responsibility to manage all of its effects which will become more difficult as more states are added
	Adding a new effect requires modifying the actor class

State Pattern

Design 1 State pattern and StatefulCreature
StatefulCreature extends Actor and encapsulates the common state machine logic including currentState, initialEntryDone, DizzyStatus checking, and the state transition flow. It also provides skipTurn() and passive() hook methods. Meeple and Swarmling extend StatefulCreature and only override their own specific logic.

pros	cons
Following the DRY principle, the state transition logic is implemented only once in `StatefulCreature`.	Added a layer of abstraction, making the class structure deeper
To implement OCP, simply inherit from `StatefulCreature` when adding a new creature.	
Satisfy SRP, each state class is responsible only for its own behavior and transition logic	

Actor Type

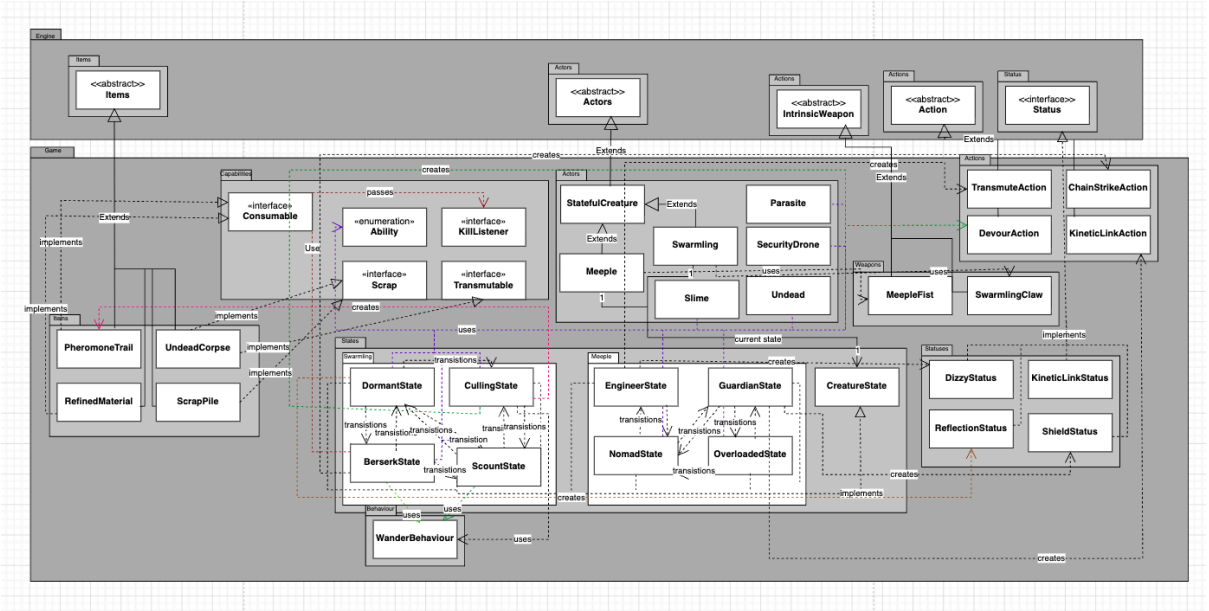
Design 1 Ability Enum Check
The state class uses `hasAbility()` to check the actor type. For Undead, it calls `enableAbility(Ability.IS_UNDEAD)`, and for Parasites, it calls

`enableAbility(Ability.IS_PARASITE)`. The state class can identify a Parasite simply by calling `actor.hasAbility(Ability.IS_PARASITE)`.

pros	cons
Satisfy DIP, state classes do not depend on any specific Actor	need to manually call `enableAbility` in each Actor's constructor.
Satisfy OCP,To add a new parasite, simply call `enableAbility(IS_PARASITE)` without modifying any state classes.	
State classes do not need to import specific Actor classes.	
New entities are automatically detected by all statuses	

Design 2: Using instanceof to identify actors within the logic

Pros	Cons
Obvious which class is being targeted	Every state must import and depend on actor classes
	If we introduce a new actor, every state that needs to check for it will need to be updated



In our implementation, we chose design 1 and design 2. The introduction of `StatefulCreature` eliminated code repetition between `Meeple` and `Swarmling`. Each state class manages its own transition logic via `checkTransitions()`, satisfying the requirements of REQ5 Rule 2 (The next state must be deterministic), Rule 4 (The condition to transition between the states must NOT be trivial), and Rule 5 (The state transition itself must have an immediate effect). Furthermore, the use of the `Ability` enumeration ensures that all state classes are completely independent of specific Actor classes. `DormantState` checks for parasites using `hasAbility(IS_PARASITE)`, and `EngineerState` checks for undead using `hasAbility(IS_UNDEAD)`, complying with the DIP principle. If new parasite or undead types are added in the future, they can be automatically recognized by all states simply by calling `enableAbility()`, without requiring any modifications to the state classes, thereby satisfying the OCP principle.